

IntraComms

LAN-Based Internal Communication System

Project Report: Final Year Project - Semester 3
Author: Ibrahim Habib
Student ID: LC000111000882
Institution: Lincoln University College
Program: Computer Software Engineering
Date: May 2026

Flask

Socket.IO

AES-256-GCM

SQLite

WebRTC

Student: Ibrahim Habib

Student ID: LC000111000882

Program: Computer Software Engineering -- Semester 3

Institution: Lincoln University College

Project Title: IntraComms -- LAN-Based Internal Communication System

Date: May 2026

Table of Contents

- Project Overview
 - Objectives
 - System Architecture
 - Technology Stack
 - Database Design
 - Security Design (CIA Triad)
 - Feature Implementation
 - File & Media Pipeline
 - Frontend Architecture
 - API Reference
 - Deployment & Configuration
 - Testing & Validation
 - Limitations & Future Work
 - Conclusion
-

1. Project Overview

IntraComms is a self-hosted, LAN-only internal communication platform built with Python and Flask. It allows users on the same local network to exchange real-time text messages and share files without any internet dependency. All data stays within the local network, making it suitable for environments that require offline operation or strict data locality.

The system supports:

- Real-time direct messaging between users over WebSockets.
- Broadcast messaging to all active users simultaneously.
- File and media sharing (images, video, audio, documents, archives).
- End-to-end encrypted (E2EE) messages and files using modern elliptic-curve cryptography.
- Peer-to-peer (P2P) browser-to-browser file transfers for large files, bypassing the server entirely.
- User account management with role-based access control (admin / user).
- A responsive web UI that works on both desktop and mobile browsers.

- HTTPS transport using a self-signed TLS certificate generated at startup.
-

2. Objectives

#	Objective	Status
1	Real-time messaging over LAN (no	Complete
2	Broadcast and direct messaging	Complete
3	User registration, login, and session	Complete
4	Role-based access control (admin /	Complete
5	File upload, storage, and download (user)	Complete
6	Media processing pipeline (photos,	Complete
7	End-to-end encryption for direct	Complete
8	End-to-end encrypted file transfer	Complete
9	P2P file transfer for very large files	Complete
10	HTTPS / TLS transport	Complete
11	Adaptive media quality based on	Complete
12	Presence indicators (online / offline)	Complete
13	Message soft-delete	Complete

3. System Architecture

3.1 High-Level Architecture

```

????????????????????????????????????????????????????????????????
?                               Local Area Network                      ?
?                               ?                                       ?
?  ?????????????????          HTTPS / WSS          ?????????????? ?
?  ? Client A ? ?????????????????????????????????? ?           ? ?
?  ? Browser  ?                                     ? Flask  ? ?
?  ?????????????????          ? Server  ? ?
?                                     ? :5000  ? ?
?  ?????????????????          HTTPS / WSS          ?           ? ?
?  ? Client B ? ?????????????????????????????????? ?           ? ?
?  ? Browser  ?                                     ?????????????? ?
?  ?????????????????          ?           ?
?           ?           ?           ?           ?
?           ? WebRTC DataChannel (P2P >100 MB)          ?           ?
?           ? (server relays signalling only)          ?           ?
?  ?????????????????          ?????????????? ?
?  ? Client C ?                                     ? SQLite  ? ?
?  ? Browser  ?                                     ? DB      ? ?
?  ?????????????????          ?????????????? ?
????????????????????????????????????????????????????????????????

```

3.2 Server Component Architecture

```

server/
??? server.py          <- Flask app factory + all routes + SocketIO handlers
??? models/
    ??? database.py    <- Shared SQLAlchemy db instance
    ??? user.py        <- User ORM model
    ??? message.py     <- Message ORM model
    ??? file_record.py <- FileRecord ORM model (file metadata)

```

The entire server runs from a single server.py file. This is intentional for a student project of this scale -- it keeps the codebase readable and avoids premature abstraction. The models/ package separates the database schema from the application logic.

3.3 Request Flow

HTTP (fallback form submit)

```

Browser POST /messages
??? register_routes() validates input
??? build_message() creates Message object
??? db.session.commit() persists to SQLite
??? emit_saved_message() pushes via Socket.IO
??? redirect to /dashboard

```

WebSocket (primary path)

```

Browser WebSocket frame: 42["message", {type:"send_message", payload:{...}}]
  ??? handle_socket_message() event handler
    ??? is_rate_limited() check (30 msg/min per user)
      ??? build_message() creates Message object
        ??? db.session.commit()
          ??? emit_saved_message()
            ??? socketio.emit() to user_{sender_id} room
            ??? socketio.emit() to user_{receiver_id} room

```

E2EE Message Path (client-side encryption)

```

Browser (sender):
  E2EE.encryptFor(recipientPub, plaintext)
    ??? ECIES (ECDH P-256 + AES-256-GCM) -> {ciphertext, ephemeral_pub, iv}
  E2EE.encryptFor(myPub, plaintext)
    ??? separate sender copy (so sender can read their own messages)
  WebSocket send -> server stores opaque ciphertext

Server: stores content as ciphertext, never sees plaintext

Browser (recipient):
  E2EE.decrypt(ephemeral_pub, iv, ciphertext)
    ??? ECDH derive key -> AES-GCM decrypt -> plaintext rendered in DOM

```

3.4 WebSocket Room Strategy

Flask-SocketIO rooms are used to route messages without broadcasting to everyone:

Room	Members	Used for
user_{id}	One user (all their tabs/devices)	Direct messages, media_ready
broadcast	All connected users	Broadcast messages, presence
On connect, each user joins both their personal room and the broadcast room.		events

3.5 Connection Quality & Adaptive Media

The client measures round-trip time (RTT) to the server via GET /ping and reports a connection tier:

Tier	Condition	Photo max	Video chunk	Audio bitrate
wired	RTT < 5 ms,	2560 px, Q88	20 MB	64k
wifi_good	RTT < 20 ms,			

downlink > 20 Mbps

1920 px, Q85

5 MB

48k

Tier	Condition	Photo max	Video chunk	Audio bitrate
wifi_weak	RTT < 60 ms,	1280 px, Q80	2 MB	32k
slow	Otherwise	800 px, Q75	1 MB	24k

The server uses the worst active tier across all of a user's connections to configure media processing.

4. Technology Stack

Layer	Technology	Version	Purpose
Web Framework	Flask	3.1.1	HTTP routing, Jinja2
Real-time	Flask-SocketIO	5.5.1	WebSocket event system
Async worker	eventlet	0.39.1	Async I/O for SocketIO
ORM	Flask-SQLAlchemy	3.1.1	Database access layer
Database	SQLite	(built-in)	Local persistent storage
Auth	Flask-Login	0.6.3	Session management,
Cryptography	cryptography	44.0.3	log, RSA, ECDH, AES-GCM,
Image processing	Pillow	>=10.0.0	WebP re-encode, EXIF
Video/Audio	FFmpeg	(external)	strip, thumbnails Transcode video to
Task scheduler	APScheduler	>=3.10.0	Background cleanup of
Security	Werkzeug	3.1.3	stale uploads, password hashing,
Environment	python-dotenv	1.1.0	.env config loading
Frontend	HTML5 + CSS3 + Vanilla	--	UI, no framework
WebRTC	Browser native	--	P2P DataChannel for large
Browser Crypto	Web Crypto API	--	file transfer, client-side ECDH +
	(SubtleCrypto)		AES-GCM

5. Database Design

5.1 Entity Relationship Diagram

```

????????????????????????????????????
?          users          ?
????????????????????????????????????
? id            PK INTEGER      ?
? username      UNIQUE NOT NULL ?
? email         UNIQUE NOT NULL ?
? password_hash NOT NULL       ?
? role          'admin'|'user'  ?
? public_key    TEXT (SPKI b64) ?
? is_active     BOOLEAN         ?
? created_at    DATETIME        ?
? last_seen     DATETIME        ?
????????????????????????????????????
? 1
? sent_messages (FK sender_id)
? received_messages (FK receiver_id)
? uploaded_files (FK uploader_id)
? N
????????????????????????????????????
?          messages        ?
????????????????????????????????????
? id            PK INTEGER      ?
? sender_id     FK -> users.id  ?
? receiver_id   FK -> users.id  ?
? content       TEXT (cipher/plain)?
? ephemeral_pub TEXT (ECDH key) ?
? iv            VARCHAR(64)     ?
? sender_copy   TEXT (own copy) ?
? sender_ephemeral_pub TEXT      ?
? sender_iv     VARCHAR(64)     ?
? is_e2ee       BOOLEAN         ?
? message_type  'direct'|'broadcast'?
? is_read       BOOLEAN         ?
? is_deleted    BOOLEAN         ?
? deleted_at    DATETIME        ?
? file_record_id FK -> file_records?
? timestamp     DATETIME        ?
????????????????????????????????????
? 1
? file_record (FK)
? 1
????????????????????????????????????
?          file_records      ?
????????????????????????????????????
? id            PK INTEGER      ?
? uploader_id   FK -> users.id  ?
? receiver_id   FK -> users.id  ?
? original_filename VARCHAR(256) ?
? stored_filename VARCHAR(256) ?
? file_size     INTEGER         ?
? mime_type     VARCHAR(128)    ?
? visibility     'shared'|'broadcast'?
? download_count INTEGER        ?

```

? is_deleted	BOOLEAN	?
--------------	---------	---

? uploaded_at	DATETIME	?
---------------	----------	---

? deleted_at DATETIME ?

? status 'ready'|'processing'|'failed'?

? thumbnail_filename VARCHAR(256) ?

? original_size BIGINT ?

? processed_size BIGINT ?

? upload_tier VARCHAR(20) ?

? media_width	INTEGER	?
---------------	---------	---

? media_height INTEGER ?

? media_duration REAL ?

? is_e2ee

BOOLEAN

?

? encryption_type VARCHAR(32) ?

? file_iv VARCHAR(64) ?

? sealed_key	TEXT	?
--------------	------	---

? key_ephemeral_pub TEXT

?

? key_iv VARCHAR(64) ?

? sender_sealed_key TEXT

?

? sender_key_ephemeral TEXT ?

? sender_key_iv VARCHAR(64) ?

? encrypted_meta TEXT ?

????????????????????????????????

5.2 Schema Migration Strategy

The application uses an idempotent migration approach -- on every startup, CREATE TABLE followed by a list of ALTER TABLE ADD COLUMN statements are executed. Each statement is wrapped in a try/except, so it silently no-ops if the column already exists. This means old databases automatically gain new columns without a separate migration tool.

6. Security Design

IntraComms is designed around the CIA Triad: Confidentiality, Integrity, and Availability.

6.1 Confidentiality

Transport Security (TLS)

At startup, the server generates a self-signed X.509 certificate (RSA-2048, SHA-256, 10-year validity) using the cryptography library. The certificate includes localhost, 127.0.0.1, and the machine's LAN IP in its Subject Alternative Names. All communication uses HTTPS / WSS, preventing passive eavesdropping on the LAN.

Password Security

User passwords are never stored in plaintext. Werkzeug's generate_password_hash / check_password_hash functions use PBKDF2-HMAC-SHA256 with a random salt.

End-to-End Encryption -- Messages (ECIES)

Each user generates a P-256 ECDH keypair in the browser using the Web Crypto API (SubtleCrypto). The private key is stored in localStorage as JWK and never sent to the server. The public key (SPKI, base64) is registered with the server on every login so other users can encrypt for the user.

For each direct message:

- The sender generates a fresh ephemeral P-256 keypair.
- ECDH key agreement between the ephemeral private key and the recipient's long-term public key produces a shared secret.
- The shared secret is used directly as an AES-256-GCM key (via SubtleCrypto deriveKey).
- The message is encrypted with a 12-byte random nonce (IV).
- The ephemeral public key and IV are stored with the ciphertext -- the recipient uses them to reconstruct the same AES key.
- The sender separately encrypts with their own public key (sender copy), so they can decrypt their own sent messages without the recipient's key.

Broadcast messages are intentionally not encrypted -- they are public by nature.

End-to-End Encrypted File Transfers

For non-video direct file uploads:

- Browser generates a random AES-256-GCM key for the file.
- File is encrypted in the browser with a 12-byte random IV.
- The file key is ECIES-sealed for the recipient and (optionally) the sender separately.
- Filename, MIME type, and size are encrypted separately (encrypted metadata).
- Only the encrypted blob and sealed key material reach the server. The server never sees the plaintext file or key.

For video files (which require server-side transcoding), after transcoding the server encrypts the output at rest using ECIES-seal with the recipient's public key.

P2P File Transfers (WebRTC)

Files larger than 100 MB are sent directly browser-to-browser via WebRTC DataChannel without touching the server (except for WebRTC signalling). Each transfer uses:

- A fresh AES-256-GCM key generated by the sender.
- The key is ECIES-sealed with the recipient's P-256 public key and sent in the WebRTC offer.
- Every 64 KB chunk is individually AES-GCM encrypted with a fresh IV before entering the DataChannel.

6.2 Integrity

- AES-256-GCM provides authenticated encryption -- any tamper to the ciphertext causes decryption to fail, and the client displays "Unable to decrypt message."
- SQLAlchemy ORM with parameterised queries prevents SQL injection.
- Werkzeug secure_filename sanitises uploaded filenames before storage.
- File extension allowlisting (ALLOWED_EXTENSIONS) prevents dangerous file types.
- A 50 MB per-file size limit is enforced both client-side and server-side.
- CSRF is mitigated by the session cookie (Flask-Login) combined with same-origin enforcement of the WebSocket connection.

6.3 Availability

- Rate limiting: 30 messages per minute per user (enforced in memory by a sliding window). Excess requests receive HTTP 429 or a RATE_LIMITED WebSocket error.
- Exponential backoff reconnect: The WebSocket client retries with delays starting at 1 s, doubling on each failure, capping at 30 s. Outgoing messages are queued during disconnection and delivered on reconnect.
- Background cleanup scheduler (APScheduler): Every 30 minutes, orphaned video chunk directories older than 2 hours are deleted, and processing file records older than 1 hour are marked failed.
- Soft deletes: Messages and files are never hard-deleted from the database -- only flagged is_deleted = True. This avoids cascading reference errors and enables potential future recovery.

6.4 Access Control

Resource	Rule
All routes	Require login (@login_required)

Resource	Rule
Admin routes (/admin/...)	Require user.role == "admin"
File download	Only uploader, receiver, or broadcast-visible files
E2EE file blob	Only uploader or receiver
Message delete	Only the sender
Admin self-deactivation	Blocked (cannot deactivate own admin account)

7. Feature Implementation

7.1 User Registration & Authentication

- Registration (POST /register): Accepts username, email, password, and an optional public key. The first user to register is automatically granted admin role. Subsequent users are regular users.
- Login (POST /login): Validates credentials and checks is_active. Sets a Flask-Login session cookie.
- Logout (POST /logout): Clears the session.
- On registration, a user_joined Socket.IO event is broadcast to all connected clients, which dynamically adds the new user to the Contacts panel without a page reload.

7.2 Dashboard & Messaging

- Dashboard (GET /dashboard): Loads the 200 most recent messages relevant to the current user (sent, received, or broadcast). Client-side JavaScript filters by active chat without additional server round-trips.
- Conversation rail: Two tabs -- Chats (existing conversations) and Contacts (all users). Switching conversations is instant (DOM show/hide).
- Character limit: 2000 characters per message (enforced client-side with live counter; auto-grow textarea).
- Message preview: Sidebar shows the last message snippet and time for each conversation, updated live by JavaScript.

7.3 Presence System

- On WebSocket connect, the user is added to the _uid_conns counter (handles multiple tabs).
- On first connection (counter goes from 0 to 1), user_online is broadcast.
- On last disconnection (counter reaches 0), user_offline is broadcast.
- On connect, the server sends an online_users snapshot listing all currently online user IDs, so the connecting client immediately knows who is online.

7.4 Admin Panel

- Lists all users with creation date and status.

- Admin can activate/deactivate any user (deactivated users cannot log in).
- Admin can change roles (promote to admin or demote to user).
- Admins cannot deactivate themselves or remove their own admin role.

7.5 Message Deletion

- Only the sender may delete a message (POST /messages/{id}/delete).
- Deletion is soft -- is_deleted = True, content not wiped.
- A message_deleted Socket.IO event is emitted to the relevant rooms so all connected clients immediately show "Message deleted" without refreshing.

7.6 Key Management API

Endpoint	Method	Description
/api/me/pubkey	POST	Register/update the current user's
/api/users/{id}/pubkey	GET	Fetch another user's public key for
		E2EE

8. File & Media Pipeline

8.1 Standard Upload (POST /upload)

```

Client selects file
  ?
Client-side MIME / size check
  ?
POST /upload (multipart/form-data)
  ?
Server: secure_filename, extension allowlist, 50 MB limit
  ?
  ??? Image (Pillow available)
  ?      -> Re-encode to WebP (EXIF stripped)
  ?      -> Generate square thumbnail (WebP)
  ?      -> Store {uuid}.webp + {uuid}_thumb.webp
  ?
  ??? Audio (FFmpeg available)
  ?      -> Re-encode to Opus OGG
  ?      -> Store {uuid}.ogg
  ?
  ??? Other (video via chunk route, or fallback)
      -> Store raw with original extension
  ?
FileRecord created -> Message created -> emit_saved_message()
```

8.2 Video Upload (Chunked, POST /upload/video/chunk)

Video files are uploaded in adaptive chunks (1-20 MB depending on connection tier) to handle large files and network interruptions:

```
Client splits video into N chunks
?
POST /upload/video/chunk (chunk_index, total_chunks, media_id)
[repeated N times]
?
Server accumulates chunks in temp dir /tmp/vid_{media_id}/
?
Last chunk received:
  FileRecord (status='processing') + Message created immediately
?
Background thread _transcode_video():
  -> Assemble chunks into single file
  -> ffprobe: extract duration
  -> ffmpeg: transcode to H.264/MP4 (CRF, scale, preset per tier)
  -> If direct message + recipient has pubkey: encrypt at rest (ECIES)
  -> FileRecord.status = 'ready'
  -> Emit 'media_ready' to relevant rooms
```

8.3 E2EE File Upload (POST /upload/e2ee)

Used for direct (non-broadcast) file transfers when E2EE is available:

```
Client browser:
  Generate random AES-256-GCM file key
  Encrypt file in memory -> encrypted blob
  ECIES-seal file key for recipient
  ECIES-seal file key for sender (own copy)
  Encrypt metadata (filename, type, size) with file key
  ?
POST /upload/e2ee (encrypted blob + sealed keys + encrypted meta)
?
Server: stores opaque blob as {hex}.enc
  FileRecord(is_e2ee=True, encryption_type='client_e2ee')
?
Client (recipient):
  GET /uploads/{id}/meta -> key material for their side
  GET /uploads/{id}/blob -> encrypted blob
  ECIES-unseal file key with own private key
  AES-GCM decrypt -> plaintext file
  Decrypt metadata -> original filename + MIME type
  Create object URL -> trigger download
```

8.4 P2P Transfer (WebRTC DataChannel, files >= 100 MB)

Sender:

```
Generate AES-256-GCM transfer key
ECIES-seal key with recipient's public key
Create RTCPeerConnection + DataChannel
Create WebRTC offer
Socket.IO 'p2p_offer' -> server relays -> recipient
?
```

Recipient:

```
Receives offer + sealed key
ECIES-unseal key with own private key
Create RTCPeerConnection, setRemoteDescription
Create answer, Socket.IO 'p2p_answer' -> server -> sender
?
```

ICE candidates exchanged via Socket.IO 'p2p_ice'

?

DataChannel established (peer-to-peer, no server involvement)

?

Sender streams file in 64 KB chunks:

```
Each chunk: AES-GCM encrypt with fresh IV -> prepend IV -> send
?
```

Recipient:

```
Each chunk: split IV + ciphertext -> AES-GCM decrypt -> buffer
On 'done' signal: reassemble Blob -> trigger browser download
```

9. Frontend Architecture

9.1 File Structure

```
static/
??? css/app.css      <- Single stylesheet, all design tokens as CSS custom properties
??? js/chat.js       <- All frontend logic (~2100 lines, no external JS libraries)

templates/
??? base.html        <- Shared navbar, Bootstrap 5, flash messages
??? index.html       <- Landing / welcome page
??? register.html    <- Registration form
??? login.html       <- Login form
??? dashboard.html   <- Main chat interface
??? admin_dashboard.html <- User management
```

9.2 JavaScript Module Structure (chat.js)

The entire frontend runs as a single IIFE (Immediately Invoked Function Expression) with no external JS framework or socket.io.js client library. The raw Engine.IO v4 (EIO4) WebSocket framing protocol is implemented manually.

Module	Description
E2EE	ECDH P-256 keypair lifecycle, ECIES encrypt/decrypt for
FileE2EE	Client-side file encryption, E2EE upload to /upload/e2ee,
P2PTransfer	decrypt, WebRTC DataChannel setup, offer/answer/ICE
Socket layer	Raw WebSocket, EIO4 ping/pong, Socket.IO connect
appendMessage()	handles message build HTML, response E2EE decrypt after
dispatch()	Routes typed Socket.IO envelopes to handlers
setActiveChat()	Switches conversation, filters DOM, updates header
filterMessages()	Hides/shows message items by chat without DOM
initPreviews()	renders sidebar snippets from rendered message list
buildMediaHtml()	Renders images, video, audio, file cards, E2EE download
measureConnection()	Pings server, classifies connection tier, reports to server buttons

9.3 EIO4 Protocol Implementation

Raw frame	Meaning	Client action
"0{...}"	Engine.IO open	Send "40" (Socket.IO connect)
"2"	Ping	Send "3" (Pong)
"40"	Socket.IO connected	Mark connected, flush queued
"42[...]"	Socket.IO event	Parse JSON, call dispatch()

Outgoing events: "42" + JSON.stringify([eventName, payload])

9.4 Theming (CSS Custom Properties)

All visual tokens are CSS variables in :root:

```
--nav-bg:    #0f172a /* dark navy navbar */
--accent:    #2563eb /* primary blue */
--bubble-in: #ffffff /* incoming message */
--bubble-out: #dbeafe /* outgoing message */
--rail-w:    320px /* sidebar width */
```

The layout is a two-panel design: `.conversation-rail` (sidebar, 320 px) and `.chat-panel` (flex remainder). On screens ≤ 680 px the sidebar and panel stack vertically with mobile show/hide toggling.

10. API Reference

HTTP Endpoints

Method	Path	Auth	Description
GET	/	No	Landing page (redirects to
GET/POST	/register	No	User registration
GET/POST	/login	No	User login
POST	/logout	Yes	Logout
GET	/dashboard	Yes	Main chat UI
POST	/messages/{id}/delete	Yes	Soft-delete a message
POST	/upload	Yes	Upload one image, audio, or
POST	/upload/video/chunk	Yes	Upload one video chunk
POST	/upload/e2ee	Yes	Upload client-side
GET	/uploads/{id}	Yes	Serve plaintext file
GET	/uploads/{id}/thumb	Yes	Serve thumbnail
GET	/uploads/{id}/blob	Yes	Serve E2EE encrypted
GET	/uploads/{id}/meta	Yes	Return E2EE key material
POST	/api/me/pubkey	Yes	Register user's public key
GET	/api/users/{id}/pubkey	Yes	Get another user's public
GET	/admin	Admin	User management panel
POST	/admin/users/{id}/activate	Admin	Activate user

Method	Path	Auth	Description
POST		Admin	Deactivate user
POST	/admin/users/{id}/role /admin/users/{id}/deactivate	Admin	Change user role
GET	/ping	No	Latency probe (returns 204)

Socket.IO Events

Client -> Server:

Event	Payload	Description
message	{type:"send_message",	Send a message
client_tier	{tier, rtt}	Report connection quality
p2p_offer	payload:{receiver_id, content, {recipient_id, offer, sealed_key, ...}}	WebRTC offer relay
p2p_answer	{recipient_id, answer}	WebRTC answer relay
p2p_ice	{recipient_id, candidate, direction, ephemeral_public_key, is_e2ee, ...}}	ICE candidate relay
p2p_decline	{recipient_id, transfer_id}	Decline P2P transfer

Server -> Client (all inside a message event, typed envelope):

Type	Payload	Description
new_message	Serialized message	New message arrived
message_deleted	{message_id}	Message was deleted
media_ready	{media_id, url, thumb_url, duration}	Video transcode complete
media_failed	{media_id, error}	Video transcode failed
user_online	{user_id}	User came online
user_offline	{user_id}	User went offline
online_users	{user_ids: [...]}	Bulk presence snapshot on connect
user_joined	{user_id, username, email,	New user registered
client_config	{tier, video_chunk_size, ...} public key}	Server-assigned upload config
p2p_offer	(forwarded from sender)	WebRTC offer
p2p_answer	(forwarded from recipient)	WebRTC answer
p2p_ice	(forwarded)	ICE candidate
p2p_declined	{transfer_id}	Transfer declined by recipient

Type	Payload	Description
error	{code, message}	Error response

11. Deployment & Configuration

11.1 Prerequisites

- Python 3.10+ (.venv with Python 3.12 is preferred)
- FFmpeg (optional -- required for video transcoding and audio re-encoding)
- Pillow (optional -- required for image WebP re-encoding and thumbnails)

11.2 Setup

```
# Create and activate virtual environment
python -m venv .venv
.venv\Scripts\Activate.ps1

# Install dependencies
pip install -r requirements.txt

# Initialise database (first run only)
python -m flask --app server/server.py init-db

# Run the server (auto-generates TLS cert on first start)
python server/server.py
```

The server starts on 0.0.0.0:5000 and is accessible from any device on the LAN at <https://<host-ip>:5000>. On first access, browsers will show a self-signed certificate warning; accepting it once stores the exception permanently.

11.3 Environment Variables

Variable	Default	Description
SECRET_KEY	dev-secret-change-me	Flask session signing key
DATABASE_URL	sqlite:///intracomms.db	SQLAlchemy connection string
FLASK_DEBUG	0	Enable debug mode and reloader

11.4 CLI Commands

Command	Description
flask --app server/server.py init-db	Create all database tables

Command	Description
flask --app server/server.py clear-db	Drop and recreate all tables (destroys data)
flask --app server/server.py promote-admin <username>	Grant admin role to a user

12. Testing & Validation

12.1 Manual Testing Performed

The following scenarios were tested manually on the local network:

Scenario	Result
Register first user -> auto-promoted to admin	Pass
Register second user -> regular user role	Pass
Login / logout session lifecycle	Pass
Send broadcast message -> appears for all connected clients	Pass
Send direct message -> appears only for sender and receiver	Pass
E2EE direct message -> ciphertext in DB, plaintext in memory	Pass
Offline queue -> send while disconnected, delivered on reconnect	Pass
Image upload -> re-encoded to WebP, thumbnail generated	Pass
Video chunk upload -> assembled, transcoded, downloaded	Pass
E2EE file upload -> encrypted blob stored, decrypted on download	Pass
Admin deactivate user -> user cannot log in	Pass
Rate limit -> 31st message in 1 minute returns 429	Pass
Message deletion -> shows "Message deleted" for both sender and receiver	Pass
Self-signed TLS -> HTTPS connection accepted after cert exception	Pass
Mobile layout -> rail/panel toggle on small screens	Pass

12.2 Security Observations

- Private keys never leave the browser; the server cannot decrypt E2EE messages.
- AES-GCM authentication tags mean tampered ciphertexts fail silently (shown as "Unable to decrypt").
- SQLAlchemy's ORM parameterises all queries, preventing SQL injection.
- `secure_filename` and extension allowlisting prevent path traversal and dangerous file execution.

13. Limitations & Future Work

13.1 Current Limitations

Limitation	Detail
No message search	Messages are filtered client-side but not searchable
No read receipts	<code>is_read</code> field exists in DB but not surfaced in the UI
No message history pagination	Dashboard loads at most 200 messages; older history is
No ratchet protocol	Each E2EE message is independently encrypted
Broadcast messages are always plaintext	Group E2EE would require a key distribution mechanism (ECIES), not a Signal-style double-ratchet; forward
Single-server	No clustering or load balancing; all state is in-process
No persistent notification	No push notifications when the browser is closed memory (rate limiter, presence, tier table)
SQLite	Suitable for a LAN prototype; would need PostgreSQL for
Video encryption at rest	Video is encrypted server-side (server sees plaintext during transcode) vs. client-E2EE for non-video files

13.2 Potential Future Improvements

- Double-ratchet key exchange -- Signal Protocol for post-compromise security.
- Group chats -- Sender Keys or MLS (Messaging Layer Security) for scalable group E2EE.
- Message search -- Full-text search index on plaintext broadcast messages; E2EE messages could be indexed client-side.
- Read receipts and typing indicators -- Socket.IO events already available.
- Voice/video calls -- WebRTC media tracks using the existing signalling infrastructure.
- Push notifications -- Web Push API with service workers for background delivery.
- PostgreSQL backend -- Drop-in via `DATABASE_URL` for multi-user concurrency.

14. Conclusion

IntraComms successfully demonstrates a production-quality architecture for a LAN-based communication system within the scope of a semester-3 final project. The system implements all core requirements: real-time messaging, file sharing, user management, and end-to-end encryption.

The security design is grounded in the CIA Triad:

- Confidentiality -- ECIES (ECDH P-256 + AES-256-GCM) for messages, client-side file encryption, HTTPS transport, and PBKDF2 password hashing.
- Integrity -- AES-GCM authentication tags, parameterised SQL, filename sanitisation, and file type allowlisting.
- Availability -- Rate limiting, reconnection with exponential backoff, background cleanup, and soft deletes.

The adaptive media pipeline (Pillow + FFmpeg) ensures that images and videos are efficiently sized for the network conditions of each client, and the WebRTC P2P pathway means that very large files transfer directly between browsers without saturating the server.

The codebase is structured to be readable and maintainable: a single `server.py` orchestrates all routes and `Socket.IO` handlers, while the `models/` package cleanly separates the database schema. This makes the project well-suited as a teaching artefact demonstrating practical web security, real-time communication, and media processing concepts.

Report generated: May 2026

Project repository: `d:\IntraComms`